OPS MATE

Docker Implementation & AWS Deployment

Technical Report

Project: Containerizing and Deploying OpsMate Service Using Docker and AWS

Technology stack: Node.js, Express, MySQL, Docker, Docker Compose, Amazon ECR, Amazon ECS, AWS Fargate, Git, GitHub

Prepared for: DevOps project submission

Report date: 10 October 2026

Status note: This report is based on the project configuration and terminal output shared during the conversation. Runtime values that were not directly verified are marked as “To be captured” rather than fabricated. Replace those fields with fresh command output and screenshots before final submission.

1. Executive Summary

This report analyzes the OpsMate containerization project. OpsMate is a small Node.js/Express web service with a MySQL integration. The Dockerfile builds the application from the Node.js 20 Alpine image, installs production dependencies, copies the application source, and starts the service on port 5000. Docker Compose defines an application service and a MySQL 8.4 service, connected over a bridge network, with a named volume for database persistence.

The project has also been configured for AWS container deployment using Amazon ECR for image storage and Amazon ECS with AWS Fargate for task execution. The source repository was pushed to GitHub. A browser request to the AWS-hosted service previously timed out, and the ECS task definition was found to map container port 80 even though the application listens on port 5000. Therefore, the cloud deployment should be considered configured but not yet verified as externally reachable.

This report deliberately distinguishes confirmed configuration from current runtime state. Docker daemon status, live resource usage, current ECR image digest, and current ECS/Fargate status must be captured from the environment before they can be reported as measured facts.

2. Project Overview and Architecture

The application exposes two endpoints:

• **GET /** — displays an OpsMate status page and attempts to connect to MySQL, create a visits table, record a visit, and show the visit count.

• **GET /health** — returns JSON indicating that the Express service is responding. This endpoint does not independently verify MySQL health.

The intended local architecture is a browser/client connecting to the OpsMate application on port 5000, with the application connecting to MySQL on port 3306 over the Compose network. The intended cloud flow is Docker image → Amazon ECR → Amazon ECS task on AWS Fargate → application endpoint, with suitable VPC networking and security group rules.

3. Implementation Details

3.1 Dockerfile

```
FROM node:20-alpine
WORKDIR /app
COPY app/package*.json ./
RUN npm install --omit=dev
COPY app/src ./src
EXPOSE 5000
CMD ["npm", "start"]
```

The Dockerfile uses Node.js 20 Alpine as the base image, sets /app as the working directory, copies package metadata, installs production dependencies, copies the source code, documents port 5000, and starts the app with npm. EXPOSE is metadata; it does not publish the port by itself. Publishing is configured in Docker Compose or in the ECS task/service networking.

3.2 Application dependencies and endpoints

The package metadata identifies the application as “opsmate” version 1.0.0. Dependencies include Express 5.1 and mysql2 3.14. The server listens on process.env.PORT or port 5000 and binds to 0.0.0.0 so the service can receive traffic from outside its own container network namespace.

3.3 Docker Compose

```
services:
mysql:
image: mysql:8.4
container_name: opsmate-mysql
environment:
MYSQL_ROOT_PASSWORD: root
MYSQL_DATABASE: opsmate
volumes:
```

```
- mysql-data:/var/lib/mysql
networks:
- opsmate-network
healthcheck:
test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-proot"]
interval: 10s
timeout: 5s
retries: 10
restart: unless-stopped

app:
build:
context: .
dockerfile: Dockerfile
container_name: opsmate-app
environment:
PORT: 5000
DB_HOST: mysql
DB_PORT: 3306
DB_USER: root
DB_PASSWORD: root
DB_NAME: opsmate
ports:
- "5000:5000"
depends_on:
mysql:
condition: service_healthy
networks:
- opsmate-network
restart: unless-stopped

networks:
opsmate-network:
driver: bridge

volumes:
mysql-data:
```

Compose configures two services, a bridge network, a MySQL health check, startup dependency, port publication, and persistent database storage. The current file uses a hard-coded root password (“root”), which is suitable only for a disposable lab and must be replaced for a secure deployment. Compose startup and DB connectivity should be verified after the final filename/configuration is settled.

4. Docker Installation Verification

Docker installation verification must be recorded from the target Ubuntu host. The shared information confirms the project was operated from an Ubuntu EC2 terminal, but the current Docker Engine version and installation check output were not supplied.

```
docker --version
docker compose version
docker info
```

Expected evidence: Docker client/Engine version, Compose version, server information, storage driver, container runtime, and any permission or daemon errors. If docker info cannot contact the daemon, inspect systemd status as described in Section 5.

5. Docker Daemon Status

```
sudo systemctl status docker --no-pager
sudo systemctl is-active docker
sudo journalctl -u docker --since "1 hour ago" --no-pager
```

The daemon is the background service that builds images and manages containers. The report cannot assert its current active/inactive status without fresh command output. If inactive, investigate journal messages before restarting it; use “sudo systemctl start docker” only after confirming that starting the service is appropriate.

6. Docker Image Information

The Dockerfile defines the intended image build. The previously shared environment showed an OpsMate image named “opsmate:1.0” with image ID beginning “23346c1fcd0e”. That historical value should be rechecked because images can be rebuilt or pruned.

```
docker images --digests
docker image inspect opsmate:1.0
docker history opsmate:1.0
```

Record the repository/tag, image ID, digest, creation date, size, base image, and exposed port. Avoid reporting an image as present if the inspect command fails.

7. Container Information

```
docker ps -a --no-trunc
docker inspect opsmate-app
docker inspect opsmate-mysql
```

The Compose configuration assigns container names opsmate-app and opsmate-mysql. Actual current status, uptime, restart count, container IDs, and port bindings require fresh output. The container name may differ if a different Compose project name or configuration was used.

8. Container Resource Usage

```
docker stats --no-stream
docker system df
free -h
df -h
```

docker stats reports current CPU, memory, network I/O, and block I/O for running containers. docker system df reports Docker disk usage. These are point-in-time measurements; capture them while the services are running and note the timestamp. No resource measurements were supplied, so this report does not invent CPU, memory, or disk figures.

9. Application Logs

```
docker logs --tail 100 opsmate-app
docker logs --tail 100 opsmate-mysql
docker logs --since 30m opsmate-app
```

The application is expected to log “OpsMate listening on port 5000” on successful startup. Review logs for startup failures, database connection errors, missing environment variables, and repeated restarts. Do not publish logs containing credentials or other secrets. The actual current log output was not provided.

10. Docker Compose Service Status

```
docker compose config
docker compose ps
docker compose logs --tail 100
curl -i http://localhost:5000/
curl -i http://localhost:5000/health
```

The Compose file defines services named app and mysql. The command output should confirm whether each service is running and whether the health check reports MySQL as healthy. A successful /health response proves the Express endpoint is responding, but not that the database is available. Test the home page and review its database status as well.

11. Amazon ECR Image Information

The AWS account ID and region previously shared were 746931976614 and ap-south-1 (Mumbai). A historical task definition referenced the ECR repository “opsmate” using an image digest beginning sha256:23346c1fcd0e. This establishes that a task definition had an ECR image reference, but it does not prove the current repository image list or that the image can presently be pulled.

```
aws sts get-caller-identity
aws ecr describe-repositories --repository-names opsmate --region ap-south-1
aws ecr describe-images --repository-name opsmate --region ap-south-1
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin
```

```
746931976614.dkr.ecr.ap-south-1.amazonaws.com
```

Record repository URI, image tags, digests, pushed timestamps, and image sizes. Run the login command only when needed and avoid exposing the token or shell tracing output. Use an IAM role or appropriately scoped credentials; do not hard-code AWS keys.

12. ECS Service Status

Previously shared ECS resources included cluster “opsmate-cluster”, service “opsmate-task-service-hpikaa30”, and task definition family “opsmate-task” revision 3. These are historical identifiers; capture current status before submission.

```
aws ecs describe-clusters --clusters opsmate-cluster --region ap-south-1
aws ecs describe-services --cluster opsmate-cluster --services opsmate-task-service-hpikaa30 --region
ap-south-1
aws ecs list-tasks --cluster opsmate-cluster --service-name opsmate-task-service-hpikaa30 --region
ap-south-1
```

Record service status, desired/running/pending task counts, deployment status, task definition revision, launch type/capacity provider, and recent service events. Do not describe the service as healthy unless running task counts and application checks support that conclusion.

13. Fargate Task Status

```
aws ecs describe-tasks --cluster opsmate-cluster --tasks TASK_ARN --region ap-south-1
aws ecs describe-task-definition --task-definition opsmate-task --region ap-south-1
```

Replace TASK_ARN with an ARN returned by list-tasks. Inspect lastStatus, desiredStatus, healthStatus if configured, stoppedReason, container exitCode, image digest, portMappings, network interface/public IP details, and log configuration. The earlier task definition mapped container port 80 even though the Node.js app listens on port 5000; this mismatch was identified as a likely contributor to the browser timeout.

14. Problems Encountered

• **GitHub HTTPS authentication:** the first push failed because GitHub does not accept account passwords for Git operations. A subsequent push succeeded, indicating a valid token/credential was supplied on retry.

• **Application endpoint timeout:** browser access to the AWS public IP returned ERR_CONNECTION_TIMED_OUT.

• **Port mapping mismatch:** application listens on port 5000 while the historical ECS task definition used container port 80.

• **Current runtime evidence incomplete:** current Docker daemon, container health, resource metrics, ECR image list, ECS service/task state, and successful public endpoint response have not been provided.

• **Development credentials:** Compose currently embeds MySQL root password “root” in the YAML. This is a security weakness outside an isolated learning environment.

• **Repository organization:** the repository had backup files and legacy Compose naming, and the professional folder layout was still being built.

15. Troubleshooting Steps Performed and Recommended

Confirmed actions from the shared context:

• Reviewed Dockerfile, Compose YAML, package.json, and server.js.

• Confirmed the app listens on 0.0.0.0 and defaults to port 5000.

• Identified the ECS port mapping mismatch (container port 80 vs application port 5000).

• Identified that public access also depends on security groups, public IP/routing, and task/network configuration.

• Recovered from an initial GitHub push authentication failure; the second push succeeded.

Recommended diagnostic sequence:

```
# Local Docker
docker --version
docker compose version
sudo systemctl is-active docker
docker compose config
docker compose up -d --build
docker compose ps
docker compose logs --tail 100
curl -i http://localhost:5000/health

# AWS
aws sts get-caller-identity
aws ecr describe-images --repository-name opsmate --region ap-south-1
aws ecs describe-services --cluster opsmate-cluster --services opsmate-task-service-hpikaa30 --region
ap-south-1
aws ecs list-tasks --cluster opsmate-cluster --service-name opsmate-task-service-hpikaa30 --region
ap-south-1
```

For ECS, align the task definition containerPort with 5000, register a new task definition revision, update the service, and wait for the deployment to stabilize. Confirm the task security group permits the intended inbound port from an appropriately restricted source; ensure the subnet routing and public IP/ALB configuration supports the desired access path. Check CloudWatch logs and ECS service events if the task stops or remains unreachable.

16. Solutions Implemented / Current Solution State

The application code, Dockerfile, and Compose configuration have been created. The Git repository was successfully pushed to GitHub. An ECR image reference and ECS/Fargate resources were configured previously. The port mismatch was diagnosed, but there is no supplied evidence that the corrected task definition was registered, the ECS service updated, or the public endpoint successfully tested. Therefore, the report marks cloud reachability as pending verification rather than claiming a completed fix.

17. Security Considerations

• Replace the hard-coded MySQL root password with a strong secret supplied through a secure mechanism. Do not commit .env files or credentials to GitHub.

• Use a non-root MySQL application account with only the required permissions instead of using root from the web app.

• Store production secrets in AWS Secrets Manager or Systems Manager Parameter Store and configure ECS task execution/task roles correctly.

• Use least-privilege IAM policies for ECR push/pull, ECS deployment, logs, and task roles.

• Restrict inbound security group rules to the required sources and ports; avoid exposing MySQL port 3306 publicly.

• Use HTTPS through an Application Load Balancer and a managed certificate for production traffic.

• Keep base images and dependencies updated; scan images for vulnerabilities and avoid unnecessary packages.

• Avoid exposing AWS credentials, ECR login tokens, passwords, and sensitive log content in screenshots or reports.

• Consider running the app as a non-root user in the image and applying container CPU/memory limits where appropriate.

18. Resource Optimization Suggestions

• Use a reproducible dependency lockfile (package-lock.json) and prefer npm ci --omit=dev for predictable production builds.

• Use a multi-stage Docker build if compilation/build steps are added; keep the runtime image minimal.

• Add an appropriate .dockerignore to reduce build context size.

• Set ECS task CPU and memory based on measured utilization; avoid overprovisioning.

• Configure CloudWatch log retention and monitor CPU, memory, restart events, and request latency.

• Use ECR lifecycle policies to remove obsolete image tags/digests after a retention period.

• Use health checks that distinguish application health from database readiness.

• Avoid running CREATE TABLE on every home-page request; move schema creation to a migration/init step and use a connection pool.

• Use an Application Load Balancer for a stable endpoint and health-based routing if the project requirements justify it.

19. Suggested Improvements

• Add a README with project overview, architecture, prerequisites, setup instructions, commands, and screenshots.

• Add scripts for Docker installation, image build, run, and cleanup; validate commands before publishing.

• Use a sample environment file with placeholder values and provide a .gitignore/.dockerignore.

• Add a proper /health/live endpoint and a separate readiness check that can verify required dependencies.

• Use parameterized queries and a pooled MySQL connection.

• Add automated tests and a CI pipeline that builds and tests the image on every push.

• Tag images with immutable versions or commit SHAs rather than relying only on latest.

• Create an architecture diagram and a report containing dated screenshots of Docker, ECR, ECS, and Fargate evidence.

• Use an Application Load Balancer or a deliberate public-IP configuration and document the network/security design.

• Confirm that the repository has no secrets, then commit the finished report and documentation.

20. Evidence Checklist for Final Submission

Report section	Evidence to capture	Status based on supplied context
Docker installation	docker --version; docker compose version	To be captured
Daemon status	systemctl status docker	To be captured
Image info	docker images --digests; image inspect	Historical image reference only; recheck
Container info	docker ps -a; inspect	To be captured
Resource usage	docker stats --no-stream; docker system df	To be captured
Application logs	docker logs output	To be captured
Compose status	docker compose ps/config	Configuration reviewed; runtime to verify
ECR	describe-images output / console screenshot	Historical digest reference; recheck
ECS service	describe-services output / screenshot	Historical resource names; recheck
Fargate task	describe-tasks output / screenshot	Port mismatch previously identified
Public app test	Browser screenshot or curl response	Previously timed out; re-test

21. Conclusion

The OpsMate project demonstrates the core stages of a container workflow: creating a Node.js service, defining a Docker image, orchestrating an application and database with Compose, and preparing the image for AWS ECR/ECS/Fargate deployment. The most important outstanding validation is to capture fresh runtime evidence and confirm that the cloud application is reachable through a correctly configured port and network path. After replacing lab credentials, completing the documentation, and validating the AWS deployment, the project will be more secure, reproducible, and suitable for a technical demonstration.

Appendix — Environment details from prior project context

Item	Previously shared value
Project directory	/home/ubuntu/OpsMate-Docker-Project
Application port	5000
Compose services	app, mysql
MySQL image	mysql:8.4
ECR region	ap-south-1 (Mumbai)
AWS account ID	746931976614
ECS cluster	opsmate-cluster
ECS service	opsmate-task-service-hpikaa30
Task definition	opsmate-task:3 (historical)
GitHub repository	vrundakoli2003-sys/Docker-project1